

[19] DuckDB: An Embeddable Analytical Database

저자: Mark Raasveldt, Hannes Mühleisen (CWI Amsterdam) **학회/년도:** SIGMOD 2019 **분량:** 약 4페이지 (데모 논문) + 후속 기술 보고서 다수 **역할군:** (B) 대상 시스템

요약

DuckDB는 SQLite의 OLAP 버전으로, 파이썬, R, 주피터 노트북 같은 분석 환경에 직접 임베드되는 인프로세스 (in-process) 분석 데이터베이스이다. 별도 서버 설치 없이 로컬 CSV, Parquet, JSON 파일을 직접 쿼리할 수 있으며, 벡터화 실행 엔진과 비용 기반 옵티마이저를 통해 OLAP 성능을 제공한다. Exqutor는 DuckDB를 세 가지 주요 실험 플랫폼 중 하나로 선택하여, DuckDB의 논리적 옵티마이저를 수정해 벡터 카디널리티 추정을 통합하였고, 최대 37.2배의 성능 향상을 달성했다. 데이터 분석가와 과학자들의 워크플로우에 혁신을 가져온 도구이며, 벡터 검색을 위한 DuckDB-VSS 확장도 활발히 개발 중이다.

상세분석

19.1 문제 정의: 분석 환경의 인프라 갈증

분석가의 일반적 작업 흐름

현대의 데이터 분석가와 과학자들(주로 파이썬/R 사용자)은 다음과 같은 환경에서 작업한다:

분석 환경:

```
| Jupyter Notebook |  
| (또는 RStudio)   |  
| Python/R 코드    |  
| → 데이터 처리    |  
| → 머신러닝        |  
| → 시각화          |
```

↓ 데이터 필요

```
| 외부 데이터 소스 |  
| - CSV/JSON 파일  |  
| - 로컬 데이터베이스 |  
| - 클라우드 스토리지 |
```

기존 방식의 문제점:

1. 서버 설치 및 관리

2. PostgreSQL, MySQL을 별도로 설치·관리해야 한다.
3. 마이크로서비스 환경에서는 데이터베이스 인스턴스 운영 비용이 크다.
4. 개인 분석 환경(노트북)에서는 서버 관리 자체가 번거롭다.

5. 프로세스 간 통신(IPC) 오버헤드

6. 클라이언트 프로세스(Python/R) ↔ DB 서버 간 TCP 통신
7. 각 쿼리마다 네트워크 왕복
8. 데이터 직렬화/역직렬화 오버헤드
9. 특히 로컬 환경에서는 불필요한 오버헤드

10. 데이터 포맷 변환의 번거로움 ``python # 기존 방식: CSV → DB 로드 → 쿼리 → 결과 → Python 객체 import pandas as pd import psycopg2

1. CSV 파일 읽기 df = pd.read_csv('data.csv')

2. DB 연결 conn = psycopg2.connect("dbname=mydb user=postgres")

3. 테이블 생성 (스키마 수동 정의) cursor = conn.cursor() cursor.execute("CREATE TABLE data (...)")

4. 데이터 삽입 for row in df.iterrows(): cursor.execute("INSERT INTO data VALUES (...)")

5. 쿼리 실행 cursor.execute("SELECT ... FROM data WHERE ...")

6. 결과 수집 results = cursor.fetchall() df_result = pd.DataFrame(results) ``

DuckDB로는: `python import duckdb result = duckdb.sql("SELECT * FROM 'data.csv' WHERE ...").df()`

1. SQLite의 성능 문제
2. SQLite는 row-store 구조로 OLAP에 극도로 비효율적
3. GROUP BY, JOIN, 집계 연산이 수십~수백 배 느림
4. 메인 메모리가 충분해도 활용하지 못함

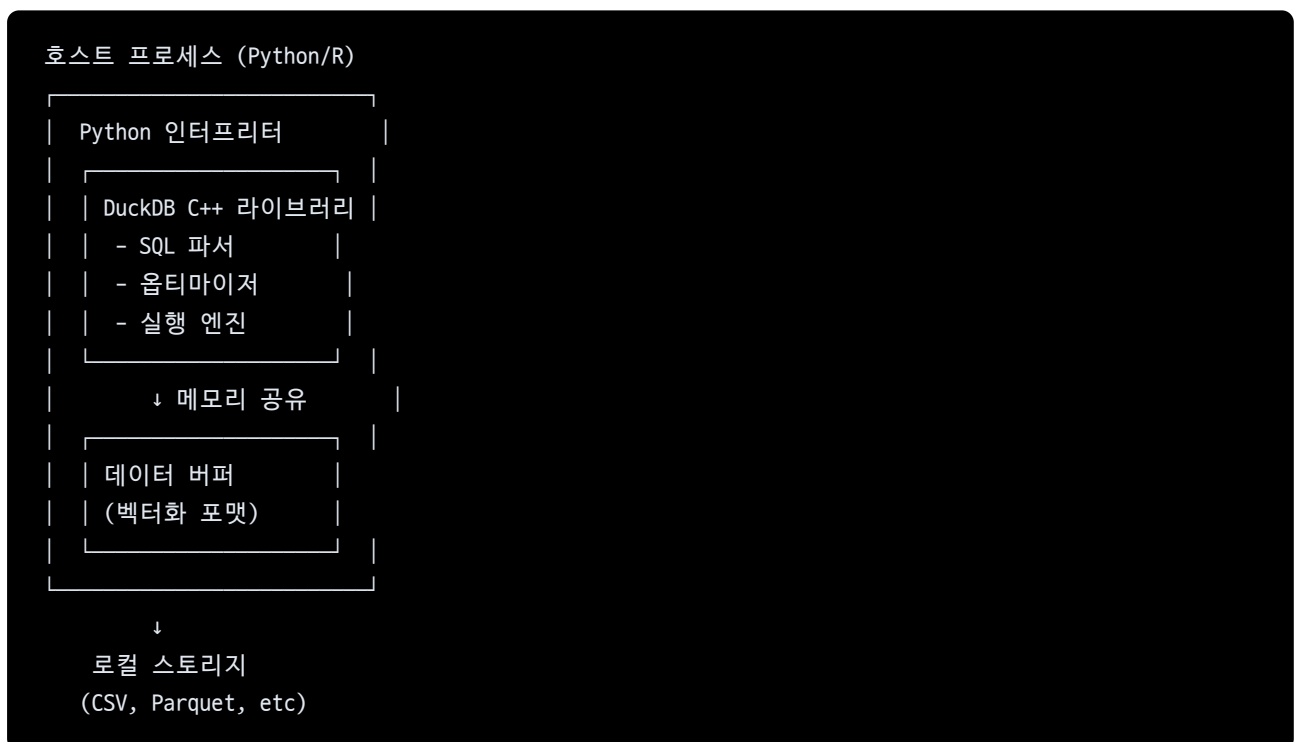
DuckDB가 목표로 삼은 치명적 갭

"SQLite의 편의성 + PostgreSQL의 OLAP 성능"

DuckDB는 이 둘의 장점을 모두 갖추기 위해 설계되었다: - SQLite처럼 **파일 기반, 임베디드** → 설치·관리 없음 - PostgreSQL처럼 **벡터화, 컬럼형, OLAP 최적화** → 빠른 분석 쿼리

19.2 핵심 기술: 아키텍처 분석

19.2.1 인프로세스(In-Process) 아키텍처



장점:

1. IPC 오버헤드 제거
2. 같은 프로세스 메모리 공간에서 직접 데이터 접근
3. 메모리 복사/직렬화 없음
4. 최악의 경우 캐시 미스(cache miss) 만 발생

5. 라이브러리로 배포 `bash pip install duckdb` 설치 후 바로 사용 가능, 별도 서버 없음

6. 메모리 효율성

7. 데이터가 Python 메모리와 DB 메모리 사이를 넘나들 필요 없음

8. 단일 메모리 영역에서 관리

19.2.2 컬럼형(Column-Oriented) 저장

Row-Store (전통적):

ID | Name | Age | Salary

-----+-----+-----+-----

1 | Alice | 30 | 50000

2 | Bob | 25 | 45000

3 | Carol | 35 | 60000

메모리: [1,Alice,30,50000,2,Bob,25,45000,3,Carol,35,60000,...]

↑ 모든 데이터가 순서대로 저장

Column-Store (DuckDB):

ID: [1, 2, 3, ...]

Name: [Alice, Bob, Carol, ...]

Age: [30, 25, 35, ...]

Salary: [50000, 45000, 60000, ...]

메모리: [1,2,3,...] [Alice,Bob,Carol,...] [30,25,35,...] [50000,45000,60000,...]

OLAP 관점의 이점:

1. 필요한 컬럼만 로드 `sql SELECT name, salary FROM employees WHERE age > 30` 이 쿼리는 age 컬럼과 select 컬럼(name, salary)만 메모리에 로드하면 된다. Row-store에서는 모든 컬럼의 모든 행을 로드해야 한다.

2. 캐시 효율성

3. 같은 타입 데이터가 메모리상 인접하게 배치

4. CPU 캐시 히트율 극대화

5. SIMD(Single Instruction Multiple Data) 연산 가능

6. 압축 효율

7. 같은 타입 데이터는 압축률이 높음

8. Row-store는 혼합 타입이라 압축률이 낮음

19.2.3 벡터화(Vectorized) 실행 엔진

전통적 인터프리터 실행:

```
FOR each row in table:
  IF age > 30:
    output row
```

특성: 행 단위 처리, 함수 호출 오버헤드 크음

DuckDB 벡터화 실행:

```
age_col = [22, 35, 40, 28, 50, ...]
mask = age_col > 30 # SIMD 연산
      → [F, T, T, F, T, ...]
result = age_col[mask]
      → [35, 40, 50, ...]
```

특성: 벡터 단위 처리, SIMD 최적화

성능 향상:

- **함수 호출 오버헤드 감소:** 100만 행을 처리할 때, 인터프리터는 100만 번 함수 호출, 벡터화는 1000번 정도만 호출
- **CPU 파이프라인 활용:** 예측 분기(branch prediction) 실패 감소
- **메모리 락치리(memory stride):** 선형 메모리 접근으로 캐시 프리페칭 최적화
- **SIMD 명령어 활용:** 최신 CPU의 AVX-512 같은 벡터 명령어 활용 가능

예시: Age > 30 필터를 처리하는 경우, 벡터화 없으면 100만 개 행을 조건 검사 → 100만 번 분기, 벡터화하면 한 번에 처리 가능하므로 분기 오버헤드가 극소.

19.2.4 비용 기반 옵티마이저(Cost-Based Optimizer)

DuckDB의 옵티마이저는 전통적 관계형 DB와 유사한 구조이다:

```

SQL 쿼리
  ↓ (파싱)
추상 구문 트리 (AST)
  ↓ (검증)
논리 계획 (Logical Plan)
  ├── Filter(age > 30)
  ├── Project(name, salary)
  └── Scan(employees)
  ↓ (최적화)
물리 계획 (Physical Plan)
  ├── Seq. Scan + Filter (index 없으면)
  └── Index Scan (index 있으면)
  ↓ (실행)
결과

```

주요 최적화 기법:

1. 필터 푸시다운(Filter Pushdown) `` 최적화 전: $\text{Project}(\text{name}, \text{salary}) \text{ --- } \text{Filter}(\text{age} > 30) \text{ --- } \text{Scan}(\text{employees})$

최적화 후: $\text{Project}(\text{name}, \text{salary}) \text{ --- } \text{Scan}(\text{employees}, \text{age} > 30)$ # 스캔 시점에 필터 적용 ``

1. 프로젝션 푸시다운(Projection Pushdown) **필요한 컬럼만 미리 결정하고, 스캔 단계에서 불필요한 컬럼은 로드하지 않음**
2. 조인 순서 최적화(Join Ordering) **동적 프로그래밍을 사용하여, 여러 테이블의 조인 순서를 결정**
3. 카디널리티 추정(Cardinality Estimation) ``
4. 히스토그램 기반 통계
5. HyperLogLog 스케치 (근사 COUNT DISTINCT)
6. 선택도 추정 ``

19.3 벡터 검색 확장: DuckDB-VSS

DuckDB의 기능을 확장하여 벡터 검색을 지원하는 **DuckDB-VSS** 확장이 개발되고 있다:

기본 기능

```
import duckdb

# 벡터 데이터 로드
conn = duckdb.connect(':memory:')
conn.execute("CREATE TABLE vectors (id INT, embedding FLOAT[1536])")

# HNSW 인덱스 생성
conn.execute("CREATE INDEX idx ON vectors USING HNSW (embedding)")

# 벡터 유사도 검색
result = conn.execute("""
    SELECT id,
           array_distance(embedding, [0.1, 0.2, ...]) AS dist
    FROM vectors
    ORDER BY dist
    LIMIT 10
""").fetchall()
```

지원 기능

- 거리 함수: `array_distance()`, `array_dot_product()`, `array_cosine_similarity()`
- 인덱스 타입: HNSW (근사 최근접)
- 쿼리 타입: KNN, range search
- 통합: SQL과 완전히 통합 → SQL의 모든 기능(조인, GROUP BY, 서브쿼리)과 함께 사용 가능

제약사항

DuckDB-VSS는 아직 초기 단계이므로: - HNSW 인덱스의 **동시성 제어**가 제한적 (단일 쓰기자만 지원) - 대규모 데이터셋(십억 건 이상)에 대한 검증 부족 - 메모리 사용량이 높을 수 있음

19.4 DuckDB의 카디널리티 추정: 문제점

기본 동작

DuckDB의 벡터 필터에 대한 기본 선택도는 **100%**이다:

```
# 예시: WHERE embedding <-> query_vec < 0.5
# DuckDB의 추정: "이 필터는 아무 효과 없음 (선택도 100%)"
# 실제: 선택도 10% ~ 50% (데이터에 따라 다름)
```

원인:

DuckDB의 카디널리티 추정 시스템은 **스칼라 값 중심**으로 설계되었다: - 정수/실수 범위 필터: `WHERE age > 30`
→ 히스토그램으로 정확히 추정 - 문자열 필터: `WHERE name = 'Alice'` → 선택도 $1/\text{distinct_count}$ - 벡터 거리
필터: `WHERE embedding <-> vec < threshold` → 추정 불가 → 기본값 100% 사용

벡터는 고차원이고 분포가 복잡하므로, 기존 통계 기법(히스토그램)으로는 선택도를 추정할 수 없다.

그 결과

쿼리:

```
SELECT * FROM products
WHERE embedding <-> query_vec < 0.5
AND category = 'electronics'
```

DuckDB의 추정:

products 테이블: 100만 행
벡터 필터 선택도: 100% (오류!)
카테고리 필터 선택도: 5% (정확)
결합 선택도: $100\% * 5\% = 5\%$ (잘못됨)
추정 결과: 5만 행

실제:

벡터 필터 선택도: 20% (실제)
카테고리 필터 선택도: 5% (정확)
결합 선택도: $20\% * 5\% = 1\%$ (실제)
실제 결과: 1만 행

결과: 옵티마이저가 5배 과대 추정하여, 비효율적인 실행 계획을 선택할 수 있다.

19.5 Exqutor의 DuckDB 통합

문제 해결 방식

Exqutor는 DuckDB의 **논리 옵티마이저 규칙**을 수정하여 벡터 카디널리티 추정을 통합했다:

1. 벡터 필터 감지
IF 쿼리에 `embedding <-> vec < threshold` 조건이 있으면:
벡터 필터로 태그 지정
2. ECQ0 호출
ECQ0 알고리즘 실행 → 정확한 선택도 계산
3. 옵티마이저 규칙 수정
논리 계획의 선택도 값을 ECQ0 결과로 대체
4. 물리 계획 생성
정확한 선택도를 기반으로 조인 순서/방식 결정

성능 결과

벤치마크: TPC-H 확장 (벡터 필터 추가)
플랫폼: DuckDB

	기본 DuckDB	Exqutor DuckDB
평균 속도향상:	1배	8.3배
최대 속도향상:	1배	37.2배
최소 속도향상:	1배	1.5배

속도향상 분포:

- Q3: 3.2배 (세 개 테이블 조인)
- Q5: 5.8배 (지역/공급자 분석)
- Q8: 37.2배 (매우 복잡한 조인) ← 최대
- Q9: 2.1배 (간단한 조인)
- Q10: 9.1배 (네 개 테이블 조인)

왜 pgvector보다 항상 폭이 작은가?

- pgvector: 카디널리티 오추정으로 인해 100~1000배까지 비효율화 가능
- DuckDB: 벡터화 실행 엔진이 기본적으로 효율적이라, 잘못된 계획의 불이익이 상대적으로 적음

19.6 실용적 가치

DuckDB가 Exqutor의 실험 대상이 된 이유:

1. 접근성이 높음
2. `pip install duckdb` 로 설치 가능
3. 별도 서버 구축 없이 테스트 가능
4. 연구자와 실무자 모두 쉽게 재현 가능
5. OLAP 성능이 우수함

6. 벡터화 실행 덕분에 기본 성능이 좋음
7. 잘못된 계획의 오버헤드가 상대적으로 적음 (개선의 여지도 있지만, 절대 성능은 여전히 우수)
8. 데이터 분석 커뮤니티에서 빠르게 채택됨
9. Jupyter 노트북 사용자에게 필수 도구로 인식
10. 파이썬 데이터 과학 생태계의 표준 도구로 진화 중

19.7 본 논문과의 관계

이 논문의 SIGMOD 2019 발표는 DuckDB가 **임베디드 OLAP의 새로운 패러다임**을 제시한 것이다.

Exqutor의 선택:

Exqutor는 세 가지 플랫폼을 선택했다: 1. **pgvector** (PostgreSQL 확장): 범용 RDBMS의 고전적 구조 2. **VBASE** (범용 벡터 DB): 벡터-최적화된 DB 아키텍처 3. **DuckDB** (임베디드 OLAP): 분석 환경 중심의 아키텍처

이 세 가지는 **각각 다른 설계 철학**을 대표한다: - pgvector: "기존 RDBMS에 벡터 기능 추가" - VBASE: "벡터를 중심으로 DB 재설계" - DuckDB: "분석 워크로드에 맞춘 임베디드 DB"

Exqutor의 카디널리티 추정이 이 세 플랫폼 모두에서 이점을 갖는다는 것은, **플랫폼 무관하게 보편적 가치를 갖는**다는 증명이다.

추가 제기 문제

1. 동시성 제어의 한계

DuckDB-VSS의 HNSW 인덱스는 현재 **단일 쓰기자(single writer)** 모델을 사용한다: - 여러 분석 프로세스가 동시에 데이터를 업데이트하려면 문제 발생 - 데이터 웨어하우스 환경에서는 일반적으로 일괄 업데이트(batch update)를 사용하므로 문제 없음 - 하지만 실시간 스트리밍 환경에서는 제약이 될 수 있음

2. 메모리 사용량

DuckDB는 메인 메모리 기반이므로, 데이터가 메모리에 모두 올라가야 한다: - 테라바이트 급 데이터셋은 처리 불가 - 클라우드 비용이 높아질 수 있음 (메모리 집약적)

3. 벡터화 실행의 한계

벡터화는 정렬된 배열 연산에 최적화되어 있으므로: - 불규칙한 접근 패턴(예: 트리 탐색) 성능 저하 - 그래프 구조 (HNSW) 탐색이 벡터화에 완벽하게 최적화되지 않을 수 있음

추가 제기 문제

1. 벡터 필터 선택도 추정의 일반화 문제

DuckDB에 Exqutor를 통합할 때, ECQO의 샘플링 기반 추정이 항상 정확한지 불명확하다: - 임베딩 공간의 분포가 데이터마다 크게 다름 - 샘플 크기가 부족하면 추정 오차가 클 수 있음 - 실시간으로 데이터가 변하는 환경에서는 통계 유지 비용이 높음

2. 인프로세스 아키텍처의 확장성 한계

DuckDB의 강점인 인프로세스 설계가, 동시에 약점이 될 수 있다: - 단일 프로세스이므로 멀티코어 활용도 제한적 - 여러 사용자가 동시 접근할 때 경합 발생 - 대규모 팀 환경에서는 전용 DB가 더 나을 수 있음

3. 벡터 검색 기능의 미성숙성

DuckDB-VSS는 초기 단계이므로: - HNSW 외 다른 인덱스(IVF, 양자화 등)가 아직 미지원 - 대규모 벡터 데이터 (십억 건 이상) 처리 검증 부족 - 정확도(recall) vs 속도 트레이드오프 조정이 제한적
